LeetCode 1115. Print FooBar Alternately: six ways

Python concurrency cheat sheet: Semaphore, Lock, Condition, Event, Barrier, Queue

Problem

Two threads share one **FooBar** object. Thread A calls **foo()** and thread B calls **bar()**, each **n** times. The output must be **"foobar"** repeated n times, so the calls must strictly alternate, foo first. The threads can start in either order. (Problem paraphrased, not the exact LeetCode text.)

Core idea

Every solution does the same thing: **foo waits for its turn, prints, then hands the turn to bar, and bar does the reverse.** The primitives differ only in how the turn is passed. All six were run with both start orders, n = 1, 5 and 200, 50 times each, and all printed foobar repeated n times.

1. Semaphore (standard answer)

```
class FooBar:
    def __init__(self, n):
        self.n = n
        self.foo_sem = threading.Semaphore(1)
        self.bar_sem = threading.Semaphore(0)

    def foo(self, printFoo):
        for _ in range(self.n):
            self.foo_sem.acquire()
            printFoo()
            self.bar_sem.release()

    def bar(self, printBar):
        for _ in range(self.n):
            self.bar_sem.acquire()
            printBar()
            self.foo_sem.release()
```

A permit is the turn token. **Semaphore(1)** means your turn now, **Semaphore(0)** means wait until released.

2. Lock

```
class FooBar:
    def __init__(self, n):
        self.n = n
        self.foo_lock = threading.Lock()
        self.bar_lock = threading.Lock()
        self.bar_lock.acquire()   # bar blocked first

    def foo(self, printFoo):
        for _ in range(self.n):
            self.foo_lock.acquire()
            printFoo()
            self.bar_lock.release()

    def bar(self, printBar):
        for _ in range(self.n):
            self.bar_lock.acquire()
            printBar()
            self.foo_lock.release()
```

Works because threading.Lock has no owner, so another thread can release it. It is a binary semaphore. RLock would not work. Say you are using it as a signal.

3. Condition

```
class FooBar:
    def __init__(self, n):
        self.n = n
        self.cond = threading.Condition()
        self.foo_turn = True

    def foo(self, printFoo):
        for _ in range(self.n):
            with self.cond:
                self.cond.wait_for(lambda: self.foo_turn)
                printFoo()
                self.foo_turn = False
                self.cond.notify_all()

    def bar(self, printBar):
        for _ in range(self.n):
            with self.cond:
                self.cond.wait_for(
                    lambda: not self.foo_turn)
                printBar()
                self.foo_turn = True
                self.cond.notify_all()
```

wait_for(pred) is shorthand for *while not pred(): wait()*, which handles spurious wakeups. Most general: for N threads use a turn counter instead of a boolean.

4. Event

```
class FooBar:
    def __init__(self, n):
        self.n = n
        self.foo_ev = threading.Event()
        self.bar_ev = threading.Event()
        self.foo_ev.set()          # foo starts

    def foo(self, printFoo):
        for _ in range(self.n):
            self.foo_ev.wait()
            self.foo_ev.clear()
            printFoo()
            self.bar_ev.set()

    def bar(self, printBar):
        for _ in range(self.n):
            self.bar_ev.wait()
            self.bar_ev.clear()
            printBar()
            self.foo_ev.set()
```

An Event stays set until cleared, so each waiter must **clear()** after waking. Safe here because exactly one thread waits on each event.

5. Barrier

```
class FooBar:
    def __init__(self, n):
        self.n = n
        self.barrier = threading.Barrier(2)

    def foo(self, printFoo):
        for _ in range(self.n):
            printFoo()
            self.barrier.wait()   # 1: foo done
            self.barrier.wait()   # 2: bar done

    def bar(self, printBar):
        for _ in range(self.n):
            self.barrier.wait()   # 1: wait for foo
            printBar()
            self.barrier.wait()   # 2: bar done
```

Cross the barrier **twice** per iteration. With one wait, foo could print the next foo while bar is still printing. Barrier is cyclic, so it resets itself.

6. Queue

```
class FooBar:
    def __init__(self, n):
        self.n = n
        self.foo_q = queue.Queue()
        self.bar_q = queue.Queue()
        self.foo_q.put(None)       # foo's token

    def foo(self, printFoo):
        for _ in range(self.n):
            self.foo_q.get()
            printFoo()
            self.bar_q.put(None)

    def bar(self, printBar):
        for _ in range(self.n):
            self.bar_q.get()
            printBar()
            self.foo_q.put(None)
```

Each queue is a mailbox for a turn token, and a blocking get() waits for it. Queue is a lock plus conditions inside, so this is a semaphore in disguise.

Comparison

Approach	Fit	Notes
Semaphore	Best	Shortest, clearest. Expected answer.
Condition	Very good	Most general. Extends to N threads with a turn counter.
Lock	OK	Binary semaphore. Valid only because Lock has no owner.
Event	OK	Needs clear() after wait. Better for one-time flags.
Queue	OK	Token passing. Natural for pipelines.
Barrier	Awkward	Two waits per iteration. Shown to demo the primitive.

Common pitfalls

- Both semaphores at 1 lets either thread go first. Both at 0 deadlocks.
- Forgetting clear() on an Event, so the next iteration runs with no turn.
- Using if instead of while when waiting on a Condition (wait_for avoids this).
- One barrier wait per iteration, which breaks the strict order.
- Using RLock as the turn token: releasing it from the other thread raises an error.

Extending it

For 1116 (Zero Even Odd), 1195 (FizzBuzz) and first/second/third problems, use one semaphore or event per thread. The first thread starts at 1, the rest at 0, and each thread releases the **next** thread's semaphore when done.